

APPWRITE

Complete Guide for Mobile Development

Backend-as-a-Service Platform

Master Backend Development

Authentication • Databases • Storage • Functions • Realtime

Mausam Kar

Full-Stack Developer

Comprehensive Documentation | 2024

Contents

1	Introduction to Appwrite	1
1.1	What is Appwrite?	1
1.2	Why Appwrite for Mobile Apps?	1
1.3	Key Features	2
2	Appwrite Architecture	3
2.1	System Architecture	3
2.2	Core Components	3
2.2.1	API Gateway	3
2.2.2	Services Layer	3
2.2.3	Docker Infrastructure	3
3	Authentication & User Management	4
3.1	Authentication Methods	4
3.2	OAuth2 Implementation	4
3.3	User Management Features	5
3.4	Authentication Flow	5
4	Databases	6
4.1	Database Overview	6
4.2	Database Hierarchy	6
4.3	Data Types	6
4.4	CRUD Operations	6
4.4.1	Create Document	6
4.4.2	Read Documents	7
4.4.3	Update Document	8
4.4.4	Delete Document	8
4.5	Queries and Filters	8
4.6	Relationships	8
5	Storage & File Management	10
5.1	Storage Overview	10
5.2	Storage Features	10
5.3	File Operations	10
5.3.1	Upload File	10
5.3.2	Get File URL	11
5.3.3	Delete File	11
5.4	Image Manipulation	11

6	Cloud Functions	12
6.1	Functions Overview	12
6.2	Supported Runtimes	12
6.3	Function Triggers	12
6.4	Function Example	12
6.5	Use Cases	13
7	Realtime	14
7.1	Realtime Overview	14
7.2	Realtime Implementation	14
7.3	Event Types	14
8	Permissions & Security	16
8.1	Permission System	16
8.2	Permission Roles	16
8.3	Permission Types	16
8.4	Setting Permissions	16
9	Setup & Configuration	18
9.1	Creating an Appwrite Account	18
9.2	Creating a Project	18
9.3	Platform Configuration	18
9.3.1	iOS Configuration	18
9.3.2	Android Configuration	18
9.4	SDK Installation	19
9.4.1	React Native	19
9.4.2	Flutter	19
9.5	SDK Initialization	19
10	Best Practices	20
10.1	Security Best Practices	20
10.2	Performance Optimization	20
10.3	Database Design	20
11	Real-World Use Cases	21
11.1	Mobile Application Types	21
11.2	Example: Real Estate App	21
12	Troubleshooting	23
12.1	Common Issues	23
12.1.1	Authentication Errors	23
12.1.2	Database Errors	23
12.1.3	Storage Errors	23
12.2	Debug Tips	23
13	Advanced Topics	25
13.1	Self-Hosting Appwrite	25
13.2	Webhooks	25
13.3	Custom Domains	25

14 Conclusion	26
14.1 Summary	26
14.2 Getting Started	26
14.3 Resources	26

1 Introduction to Appwrite

1.1 What is Appwrite?

Definition

Appwrite is an open-source, self-hosted Backend-as-a-Service (BaaS) platform that provides developers with a set of APIs and tools to build secure and scalable applications faster. It eliminates the need to build backend infrastructure from scratch.

Appwrite provides ready-to-use backend services including:

-  **Authentication** - User management and secure login
-  **Databases** - NoSQL document databases
-  **Storage** - File storage and CDN
-  **Functions** - Serverless cloud functions
-  **Realtime** - WebSocket connections
-  **Localization** - Multi-language support

1.2 Why Appwrite for Mobile Apps?

Advantage	Benefit
Cross-Platform SDKs	Native support for iOS, Android, Flutter, React Native
Secure by Default	Built-in security features and encryption
Scalable Architecture	Auto-scaling and load balancing
Developer Friendly	Clean APIs and comprehensive documentation
Open Source	Free, transparent, and community-driven
Self-Hosted Option	Full control over your data

Table 1.1: Appwrite Advantages for Mobile Development

1.3 Key Features

Key Point

Appwrite provides over **18+** **products and services** including:

- Account Management
- OAuth2 Providers
- Team Collaboration
- Database Collections
- File Storage
- Cloud Functions
- Webhooks
- Realtime Updates
- Health Monitoring
- Locale Detection
- Avatar Generation
- Email/SMS

2

Appwrite Architecture

2.1 System Architecture

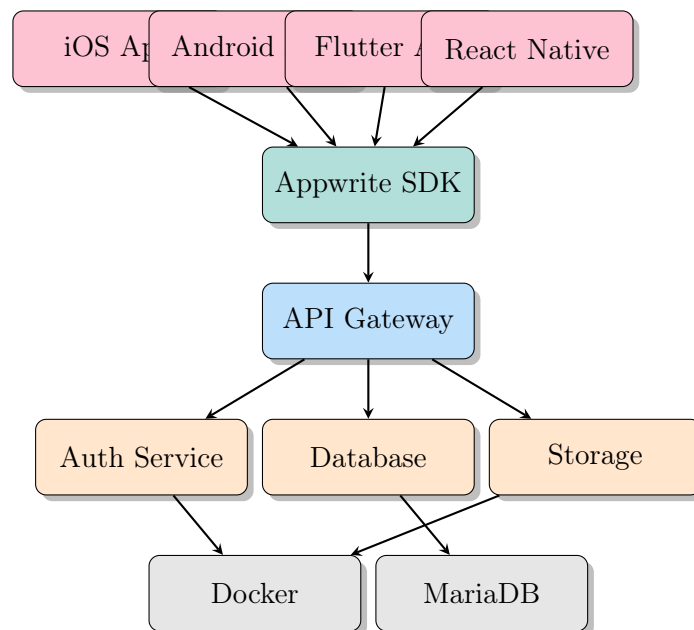


Figure 2.1: Appwrite System Architecture

2.2 Core Components

2.2.1 API Gateway

Central entry point that routes requests to appropriate services, handles authentication, and enforces security policies.

2.2.2 Services Layer

Individual microservices for authentication, database, storage, functions, and more.

2.2.3 Docker Infrastructure

Containerized deployment for easy scaling and management.

3 Authentication & User Management

3.1 Authentication Methods

Appwrite supports multiple authentication methods:

Method	Description	Use Case
Email/Password	Traditional email and password	Standard apps
Magic URL	Passwordless email link	User convenience
Phone (SMS)	OTP via SMS	Mobile-first apps
Anonymous	Temporary anonymous sessions	Guest access
OAuth2	30+ providers (Google, Facebook, etc.)	Social login
JWT	Custom token authentication	Enterprise apps

Table 3.1: Appwrite Authentication Methods

3.2 OAuth2 Implementation

```
1 import { Client, Account, OAuthProvider } from "appwrite";
2
3 const client = new Client()
4   .setEndpoint('https://cloud.appwrite.io/v1')
5   .setProject('YOUR_PROJECT_ID');
6
7 const account = new Account(client);
8
9 // Initiate Google OAuth
10 async function loginWithGoogle() {
11   try {
12     await account.createOAuth2Session(
13       OAuthProvider.Google,
14       'https://yourapp.com/success',
15       'https://yourapp.com/failure'
16     );
17   } catch (error) {
18     console.error('Login failed:', error);
19   }
20 }
```


Listing 3.1: Google OAuth Implementation

3.3 User Management Features

- **User Registration** - Create new user accounts
- **Email Verification** - Verify user email addresses
- **Password Recovery** - Reset forgotten passwords
- **Session Management** - Handle active user sessions
- **User Preferences** - Store user-specific settings
- **Account Deletion** - Allow users to delete accounts

3.4 Authentication Flow

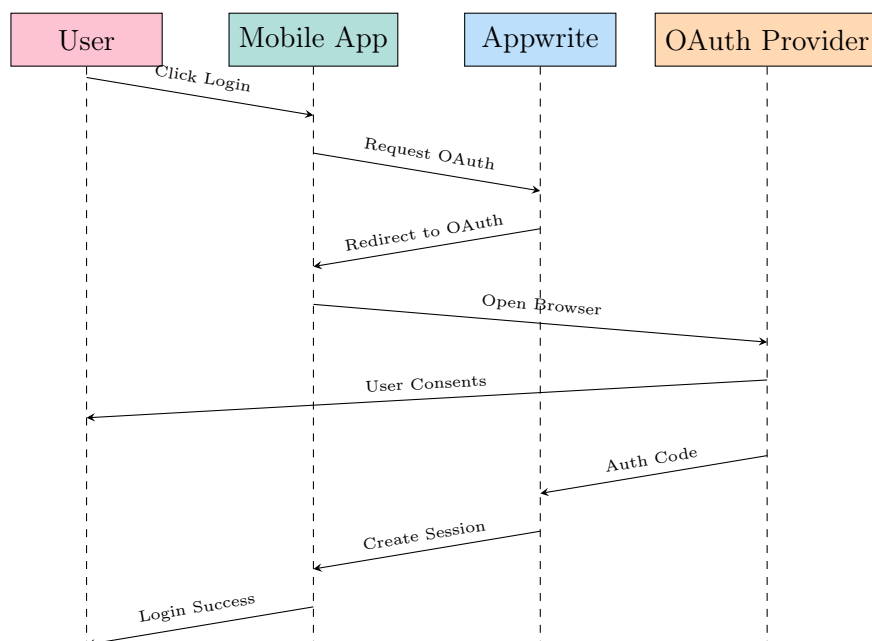


Figure 3.1: OAuth Authentication Sequence

4

Databases

4.1 Database Overview

Definition

Appwrite Databases is a NoSQL document database service that allows you to store and query structured data. It provides a flexible schema with support for relationships, queries, and real-time updates.

4.2 Database Hierarchy

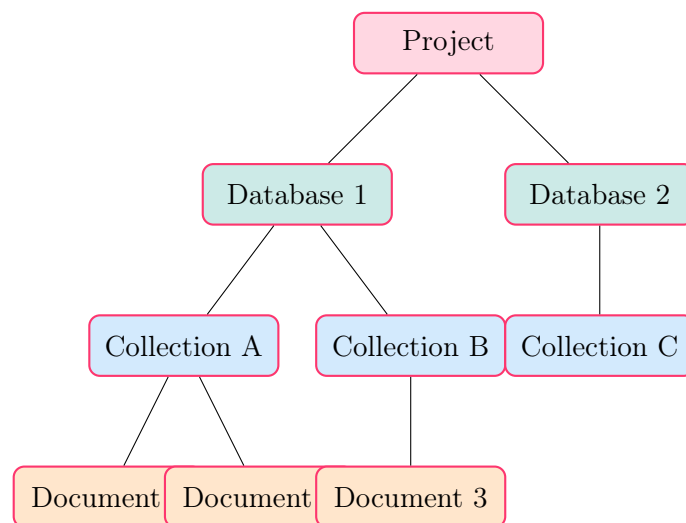


Figure 4.1: Database Hierarchy Structure

4.3 Data Types

4.4 CRUD Operations

4.4.1 Create Document

```
1 import { Client, Databases, ID } from "appwrite";
```

Type	Description	Example
String	Text data	"John Doe"
Integer	Whole numbers	42
Float	Decimal numbers	3.14159
Boolean	True/False values	true
DateTime	Date and time	2024-12-17T10:00:00
Email	Email addresses	user@example.com
URL	Web addresses	https://example.com
IP Address	IP addresses	192.168.1.1
Enum	Predefined values	["active", "pending"]
Relationship	Link to other documents	Document ID

Table 4.1: Supported Data Types

```

2
3 const databases = new Databases(client);
4
5 const document = await databases.createDocument(
6   'DATABASE_ID',
7   'COLLECTION_ID',
8   ID.unique(),
9   {
10     name: 'Luxury Villa',
11     price: 500000,
12     bedrooms: 4,
13     available: true
14   }
15 );

```

Listing 4.1: Creating a Document

4.4.2 Read Documents

```

1 import { Query } from "appwrite";
2
3 // Get all documents
4 const documents = await databases.listDocuments(
5   'DATABASE_ID',
6   'COLLECTION_ID'
7 );
8
9 // Query with filters
10 const filtered = await databases.listDocuments(
11   'DATABASE_ID',
12   'COLLECTION_ID',
13   [
14     Query.equal('available', true),
15     Query.greaterThan('price', 100000),
16     Query.orderDesc('price'),
17     Query.limit(10)
18   ]
19 );

```

Listing 4.2: Querying Documents

4.4.3 Update Document

```
1 const updated = await databases.updateDocument(  
2   'DATABASE_ID',  
3   'COLLECTION_ID',  
4   'DOCUMENT_ID',  
5   {  
6     price: 450000,  
7     available: false  
8   }  
9 );
```

Listing 4.3: Updating a Document

4.4.4 Delete Document

```
1 await databases.deleteDocument(  
2   'DATABASE_ID',  
3   'COLLECTION_ID',  
4   'DOCUMENT_ID'  
5 );
```

Listing 4.4: Deleting a Document

4.5 Queries and Filters

Query Method	Description
Query.equal()	Match exact value
Query.notEqual()	Don't match value
Query.lessThan()	Less than value
Query.greaterThan()	Greater than value
Query.search()	Full-text search
Query.orderAsc()	Sort ascending
Query.orderDesc()	Sort descending
Query.limit()	Limit results
Query.offset()	Skip results
Query.cursorAfter()	Pagination cursor

Table 4.2: Query Methods

4.6 Relationships

Appwrite supports three types of relationships:

1. **One-to-One** - Single document relates to single document
2. **One-to-Many** - Single document relates to multiple documents
3. **Many-to-Many** - Multiple documents relate to multiple documents

```
1 // Create a property with agent relationship
2 const property = await databases.createDocument(
3   'DATABASE_ID',
4   'PROPERTIES',
5   ID.unique(),
6   {
7     name: 'Modern Apartment',
8     agent: 'AGENT_DOCUMENT_ID', // Relationship
9     reviews: ['REVIEW_1', 'REVIEW_2'] // Multiple relationships
10  }
11 );
12
13 // Fetch with related data
14 const propertyWithAgent = await databases.getDocument(
15   'DATABASE_ID',
16   'PROPERTIES',
17   'PROPERTY_ID'
18 );
19 // Agent data is automatically included
```

Listing 4.5: Using Relationships

5

Storage & File Management

5.1 Storage Overview

Key Point

Appwrite Storage provides secure file storage with built-in CDN, image manipulation, and virus scanning. Perfect for storing user uploads, images, videos, and documents.

5.2 Storage Features

- **File Upload** - Support for files up to 5GB
- **Image Manipulation** - Resize, crop, compress on-the-fly
- **Virus Scanning** - ClamAV integration
- **File Preview** - Generate thumbnails
- **CDN Integration** - Fast global delivery
- **Compression** - Automatic file compression

5.3 File Operations

5.3.1 Upload File

```
1 import { Storage, ID } from "appwrite";
2
3 const storage = new Storage(client);
4
5 // Upload file
6 const file = await storage.createFile(
7   'BUCKET_ID',
8   ID.unique(),
9   document.getElementById('file-input').files[0],
10  ['read("any")'] // Permissions
11 );
12
13 console.log('File uploaded:', file.$id);
```

Listing 5.1: Uploading a File

5.3.2 Get File URL

```

1 // Get file view URL
2 const url = storage.getFileView('BUCKET_ID', 'FILE_ID');
3
4 // Get file download URL
5 const downloadUrl = storage.getFileDownload('BUCKET_ID', 'FILE_ID');
6
7 // Get file preview with manipulation
8 const preview = storage.getFilePreview(
9     'BUCKET_ID',
10    'FILE_ID',
11    400,    // width
12    300,    // height
13    'center', // gravity
14    90,     // quality
15    0,     // border width
16    'FFFFFF', // border color
17    0,     // border radius
18    1,     // opacity
19    0,     // rotation
20    'FFFFFF', // background
21    'webp' // output format
22 );

```

Listing 5.2: Getting File URL

5.3.3 Delete File

```

1 await storage.deleteFile('BUCKET_ID', 'FILE_ID');

```

Listing 5.3: Deleting a File

5.4 Image Manipulation

Parameter	Description	Values
width	Image width in pixels	0-4000
height	Image height in pixels	0-4000
gravity	Crop position	center, top, bottom, etc.
quality	Image quality	0-100
borderWidth	Border width	0-100
borderRadius	Corner radius	0-4000
output	Output format	jpg, png, gif, webp

Table 5.1: Image Manipulation Parameters

6

Cloud Functions

6.1 Functions Overview

Definition

Appwrite Functions are serverless compute solutions that allow you to run custom backend code in response to events or HTTP requests. They execute in isolated Docker containers with support for 15+ programming languages.

6.2 Supported Runtimes

- Node.js
- Python
- PHP
- Ruby
- Dart
- Deno
- Java
- Swift
- Kotlin
- .NET
- C++
- Go

6.3 Function Triggers

Trigger Type	Description
HTTP	Execute via HTTP request
Event	Trigger on database/storage/auth events
Schedule	Run on cron schedule

Table 6.1: Function Trigger Types

6.4 Function Example

```
1 // Send welcome email when user registers
2 export default async ({ req, res, log, error }) => {
3   const userId = req.body.userId;
4   const email = req.body.email;
5 }
```



```
6     try {
7         // Send welcome email logic
8         log('Sending welcome email to ${email}');
9
10        // Your email sending code here
11
12        return res.json({
13            success: true,
14            message: 'Welcome email sent'
15        });
16    } catch (err) {
17        error('Failed to send email: ' + err.message);
18        return res.json({
19            success: false,
20            error: err.message
21        }, 500);
22    }
23 };
```

Listing 6.1: Node.js Function Example

6.5 Use Cases

1. **Payment Processing** - Handle Stripe/PayPal webhooks
2. **Email Notifications** - Send transactional emails
3. **Data Processing** - Process uploaded files
4. **Third-Party Integration** - Connect to external APIs
5. **Scheduled Tasks** - Database cleanup, reports
6. **Custom Business Logic** - Complex calculations

7

Realtime

7.1 Realtime Overview

Key Point

Appwrite Realtime enables WebSocket connections to subscribe to events and receive live updates when data changes. Perfect for chat apps, notifications, and collaborative features.

7.2 Realtime Implementation

```
1 import { Client } from "appwrite";
2
3 const client = new Client()
4   .setEndpoint('https://cloud.appwrite.io/v1')
5   .setProject('YOUR_PROJECT_ID');
6
7 // Subscribe to document updates
8 client.subscribe('databases.DATABASE_ID.collections.COLLECTION_ID.
9   documents', response => {
10   console.log('Event:', response.events);
11   console.log('Payload:', response.payload);
12 });
13
14 // Subscribe to specific document
15 client.subscribe('databases.DATABASE_ID.collections.COLLECTION_ID.
16   documents.DOCUMENT_ID', response => {
17   console.log('Document updated:', response.payload);
18 });
19
20 // Subscribe to account events
21 client.subscribe('account', response => {
22   console.log('Account event:', response);
23 });
```

Listing 7.1: Subscribe to Realtime Events

7.3 Event Types

Event	Description
*.create	Document/file created
*.update	Document/file updated
*.delete	Document/file deleted
account.*	Account events
databases.*.*	Database events
storage.*.*	Storage events
functions.*.*	Function execution events

Table 7.1: Realtime Event Types

8

Permissions & Security

8.1 Permission System

Appwrite uses a role-based permission system for fine-grained access control.

8.2 Permission Roles

Role	Description
any()	Anyone (authenticated or guest)
guests()	Unauthenticated users
users()	Any authenticated user
user(ID)	Specific user by ID
team(ID)	All team members
member(ID)	Specific team member

Table 8.1: Permission Roles

8.3 Permission Types

- **Read** - View documents/files
- **Create** - Create new documents/files
- **Update** - Modify existing documents/files
- **Delete** - Remove documents/files

8.4 Setting Permissions

```
1 import { Permission, Role } from "appwrite";
2
3 // Create document with permissions
4 const document = await databases.createDocument(
5   'DATABASE_ID',
6   'COLLECTION_ID',
7   ID.unique(),
8   { title: 'My Document' },
```

```
9      [
10        Permission.read(Role.any()),
11        Permission.update(Role.user('USER_ID')),
12        Permission.delete(Role.user('USER_ID'))
13      ]
14    );
15
16    // File with team permissions
17    const file = await storage.createFile(
18      'BUCKET_ID',
19      ID.unique(),
20      fileInput.files[0],
21      [
22        Permission.read(Role.team('TEAM_ID')),
23        Permission.delete(Role.team('TEAM_ID', 'owner'))
24      ]
25    );
```

Listing 8.1: Permission Examples

9

Setup & Configuration

9.1 Creating an Appwrite Account

1. Visit <https://cloud.appwrite.io>
2. Click "Sign Up" or "Get Started"
3. Choose sign-up method (Email, GitHub, etc.)
4. Verify your email address
5. Complete your profile

9.2 Creating a Project

1. Log in to Appwrite Console
2. Click "Create Project"
3. Enter project name
4. Click "Create"
5. Note your Project ID

9.3 Platform Configuration

9.3.1 iOS Configuration

```
1 # Add bundle identifier
2 Bundle ID: com.yourcompany.yourapp
3 Name: Your App Name
```

Listing 9.1: iOS Setup

9.3.2 Android Configuration

```
1 # Add package name
2 Package Name: com.yourcompany.yourapp
3 Name: Your App Name
```

Listing 9.2: Android Setup

9.4 SDK Installation

9.4.1 React Native

```
1 npm install appwrite
2 # or
3 yarn add appwrite
```

Listing 9.3: Install Appwrite SDK

9.4.2 Flutter

```
1 flutter pub add appwrite
```

Listing 9.4: Flutter Installation

9.5 SDK Initialization

```
1 import { Client, Account, Databases, Storage } from 'appwrite';
2
3 const client = new Client();
4
5 client
6   .setEndpoint('https://cloud.appwrite.io/v1')
7   .setProject('YOUR_PROJECT_ID');
8
9 const account = new Account(client);
10 const databases = new Databases(client);
11 const storage = new Storage(client);
12
13 export { client, account, databases, storage };
```

Listing 9.5: Initialize Appwrite Client

10 Best Practices

10.1 Security Best Practices

1. **Use Environment Variables** - Never hardcode API keys
2. **Implement Proper Permissions** - Use least privilege principle
3. **Enable 2FA** - For admin accounts
4. **Regular Audits** - Review access logs
5. **HTTPS Only** - Always use secure connections

10.2 Performance Optimization

- **Use Queries Efficiently** - Add indexes for frequently queried fields
- **Implement Pagination** - Use limit and offset
- **Cache Static Data** - Reduce unnecessary API calls
- **Optimize Images** - Use proper dimensions and formats
- **Batch Operations** - Group multiple operations when possible

10.3 Database Design

Note

Schema Design Tips:

- Keep documents focused and minimal
- Use relationships appropriately
- Create indexes for search fields
- Plan for scalability from the start
- Document your schema

11 Real-World Use Cases

11.1 Mobile Application Types

App Type	Appwrite Features Used
Social Media	Auth, Database, Storage, Realtime
E-Commerce	Auth, Database, Storage, Functions
Chat Application	Auth, Database, Realtime
Task Management	Auth, Database, Realtime
Food Delivery	Auth, Database, Storage, Functions, Realtime
Real Estate	Auth, Database, Storage, OAuth
Education Platform	Auth, Database, Storage, Functions

Table 11.1: Application Types and Features

11.2 Example: Real Estate App

```
1 // 1. Setup Collections
2 // Properties: name, type, price, bedrooms, bathrooms, agent
3 // Agents: name, email, phone, avatar
4 // Reviews: rating, comment, property, user
5
6 // 2. Authentication
7 const login = async () => {
8   await account.createOAuth2Session(
9     OAuthProvider.Google,
10    successUrl,
11    failureUrl
12  );
13 };
14
15 // 3. Fetch Properties
16 const getProperties = async (type) => {
17   return await databases.listDocuments(
18     'DATABASE_ID',
19     'PROPERTIES',
20     type ? [Query.equal('type', type)] : []
21   );
22 };
23
```

```
24 // 4. Upload Property Image
25 const uploadImage = async (file) => {
26     return await storage.createFile(
27         'BUCKET_ID',
28         ID.unique(),
29         file
30     );
31 };
32
33 // 5. Subscribe to New Properties
34 client.subscribe('databases.*.collections.PROPERTIES.documents',
35     response => {
36         if (response.events.includes('create')) {
37             showNotification('New property available!');
38         }
39     }
40 );
```

Listing 11.1: Complete Real Estate Implementation

12 Troubleshooting

12.1 Common Issues

12.1.1 Authentication Errors

Warning

401 Unauthorized

- Check if OAuth provider is enabled in console
- Verify redirect URLs are correctly configured
- Ensure session is valid

12.1.2 Database Errors

Warning

Document Not Found

- Verify document ID is correct
- Check read permissions
- Ensure collection exists

12.1.3 Storage Errors

Warning

File Upload Failed

- Check file size (max 5GB)
- Verify bucket permissions
- Ensure valid file type

12.2 Debug Tips

1. **Enable Console Logging** - Monitor API requests

2. **Check Network Tab** - View request/response
3. **Review Logs** - Check Appwrite console logs
4. **Test Permissions** - Use console to test access
5. **Validate Data** - Ensure proper data structure

13 Advanced Topics

13.1 Self-Hosting Appwrite

```
1 # Install Appwrite
2 docker run -it --rm \
3     --volume /var/run/docker.sock:/var/run/docker.sock \
4     --volume "$(pwd)/appwrite:/usr/src/code/appwrite:rw \
5     --entrypoint="install" \
6     appwrite/appwrite:1.4.0
7
8 # Start Appwrite
9 docker compose up -d
```

Listing 13.1: Self-Host with Docker

13.2 Webhooks

Configure webhooks to receive HTTP callbacks when events occur:

```
1 {
2     "events": ["databases.*.collections.*.documents.*.create"],
3     "userId": "user123",
4     "payload": {
5         "$id": "doc123",
6         "name": "New Property",
7         "price": 250000
8     }
9 }
```

Listing 13.2: Webhook Payload Example

13.3 Custom Domains

Set up custom domains for your Appwrite instance:

1. Configure DNS records
2. Add domain in Appwrite console
3. Enable SSL certificate
4. Update SDK endpoint

14 Conclusion

14.1 Summary

Appwrite is a comprehensive Backend-as-a-Service platform that simplifies mobile app development by providing:

- **Complete Authentication** - Multiple providers, secure sessions
- **Powerful Databases** - NoSQL with relationships and queries
- **File Storage** - With CDN and image manipulation
- **Cloud Functions** - Serverless compute
- **Realtime Updates** - WebSocket subscriptions
- **Robust Security** - Fine-grained permissions

14.2 Getting Started

Key Point

Start building your mobile app today:

1. Create free account at <https://cloud.appwrite.io>
2. Set up your first project
3. Install SDK in your mobile app
4. Build amazing features!

14.3 Resources

- Official Documentation: <https://appwrite.io/docs>
- GitHub Repository: <https://github.com/appwrite/appwrite>
- Discord Community: <https://appwrite.io/discord>
- YouTube Tutorials: <https://youtube.com/@appwrite>

Happy Building with Appwrite!

Build like a team of hundreds.